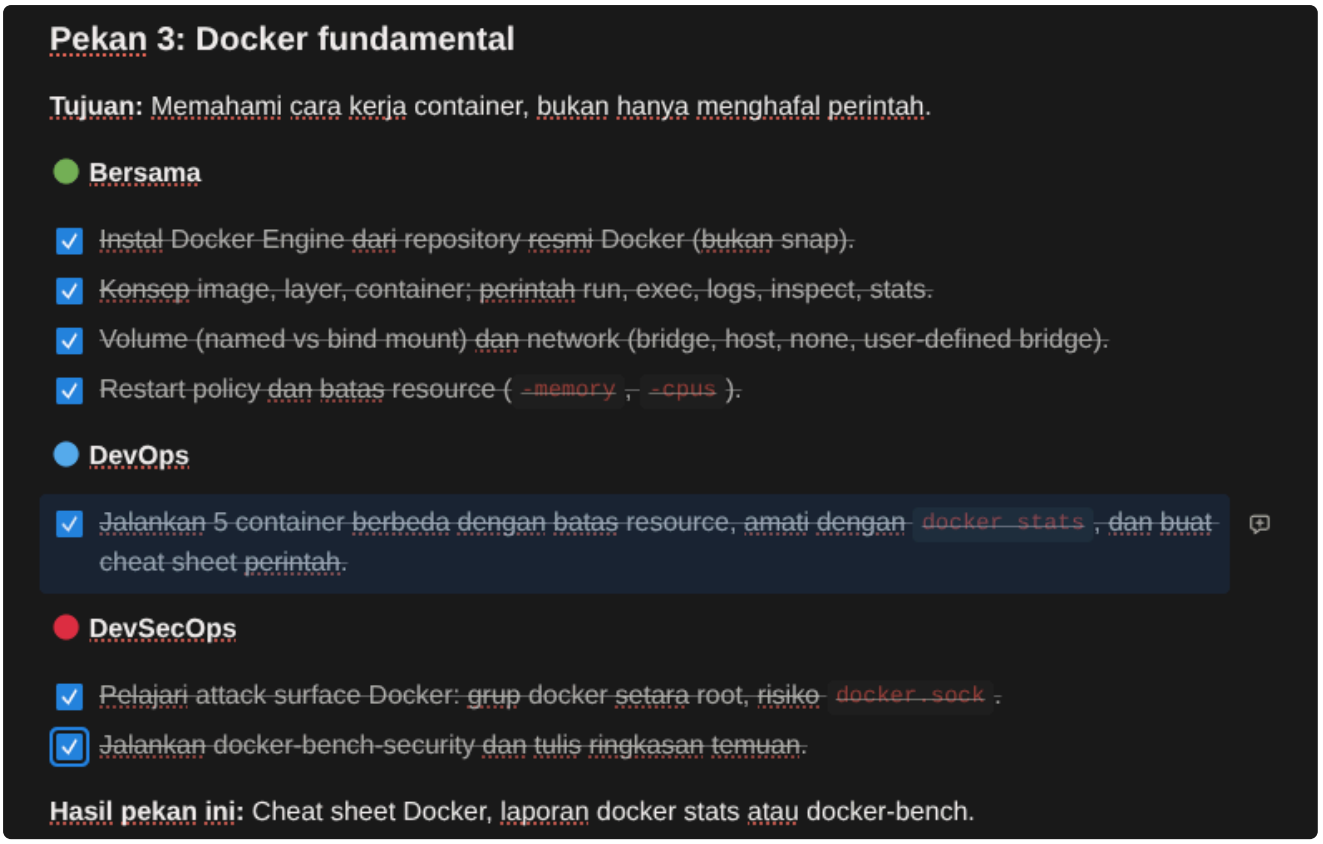
aku akan menulis dokumentasi task 3 yang mana ini sebenernya banyak praktek dan ini akan mencakup hal hal yang ada pada gambar ini



langsung aja untuk install docker engine, kita ke https://docs.docker.com/engine/install/ubuntu/ lalu kita lakukan step by stepnya dan karena vm yang aku pakai ini masih fresh, makanya aku langsung aja install dengan apt yang mana docker menyediakan command sekali jalan jadi, yaitu:

Add Docker's official GPG key:

```
sudo apt update

sudo apt install ca-certificates curl

sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL [https://download.docker.com/linux/ubuntu/gpg]
(https://download.docker.com/linux/ubuntu/gpg) -o
/etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Add the repository to Apt sources:

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF

Types: deb

URIs: [https://download.docker.com/linux/ubuntu]
(https://download.docker.com/linux/ubuntu)

Suites: (. /etc/os-release && echo "{UBUNTU_CODENAME:-$VERSION_CODENAME}")

Components: stable

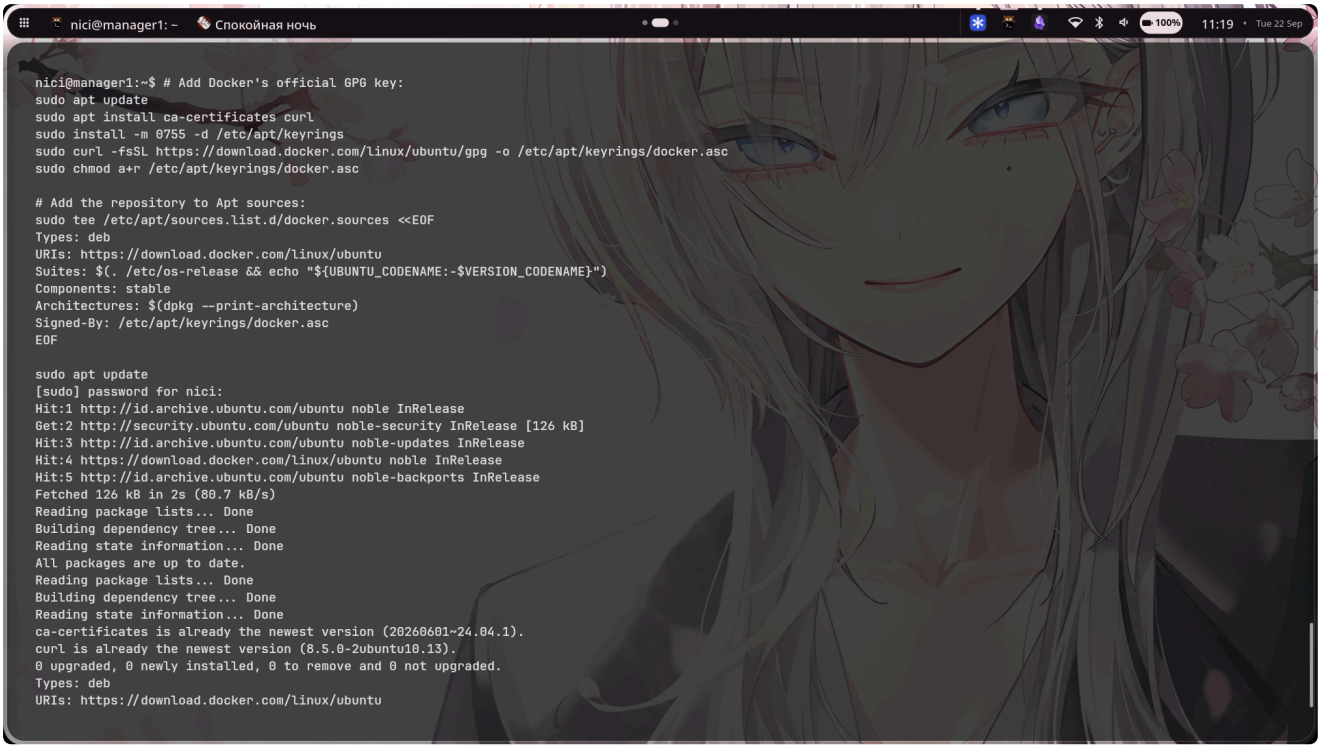
Architectures: $(dpkg --print-architecture)

Signed-By: /etc/apt/keyrings/docker.asc

EOF

sudo apt update
```

lalu jika dijalankan hasilnya akan menjadi seperti ini



lalu kita tinggal install aja docker nya dengan command

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

```
nici@manager1:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker-ce is already the newest version (5:29.8.1-1~ubuntu.24.04~noble).
docker-ce-cli is already the newest version (5:29.8.1-1~ubuntu.24.04~noble).
containerd.io is already the newest version (2.3.5-1~ubuntu.24.04~noble).
docker-buildx-plugin is already the newest version (0.37.1-1~ubuntu.24.04~noble).
docker-compose-plugin is already the newest version (5.5.1-1~ubuntu.24.04~noble).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
nici@manager1:~$
```

karena di laptop ku sudah terinstall maka hasilnya seperti itu

dan point 1 yang masalah install docker itu udah kelar lalu tinggal point kedua yang mana soal image, layer, container; perintah run, exec, logs, inspect, stats yang mana kita akan praktek mulai dari jalanin container dengan command

```
docker run -d --name web-test nginx
```

lalu

```
docker ps
```

hasilnya adalah

```
nici@manager1 ~> docker run -d --name web-test nginx

73f69e87b61867a29dff191e86294087f2296589c6ff5fa9bcf685dd74697082
nici@manager1 ~> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED          STATUS          PORTS      NAMES
73f69e87b618   nginx      "/docker-entrypoint.…"   10 seconds ago   Up 9 seconds    80/tcp     web-test
05c4b219550c   postgres   "docker-entrypoint.s…"   2 hours ago      Up 2 hours      5432/tcp   c3-postgres
122f11335f1e   redis      "docker-entrypoint.s…"   2 hours ago      Up 2 hours      6379/tcp   c2-redis
189285609fc9   nginx      "/docker-entrypoint.…"   2 hours ago      Up 2 hours      80/tcp     c1-nginx
nici@manager1 ~>
```

lihat? terbuat satu container 10 detik lalu dan kita juga akan masuk ke shell nya dengan command

```
docker exec -it web-test bash
```

```
nici@manager1 ~> docker exec -it web-test bash
root@a6b83c0f4533:/#
```

lalu ketik

```
exit
```

untuk keluar dari shell, lalu kita akan mencoba melihat log dengan command

```
docker logs web-test
```

maka hasilnya akan keluar log seperti

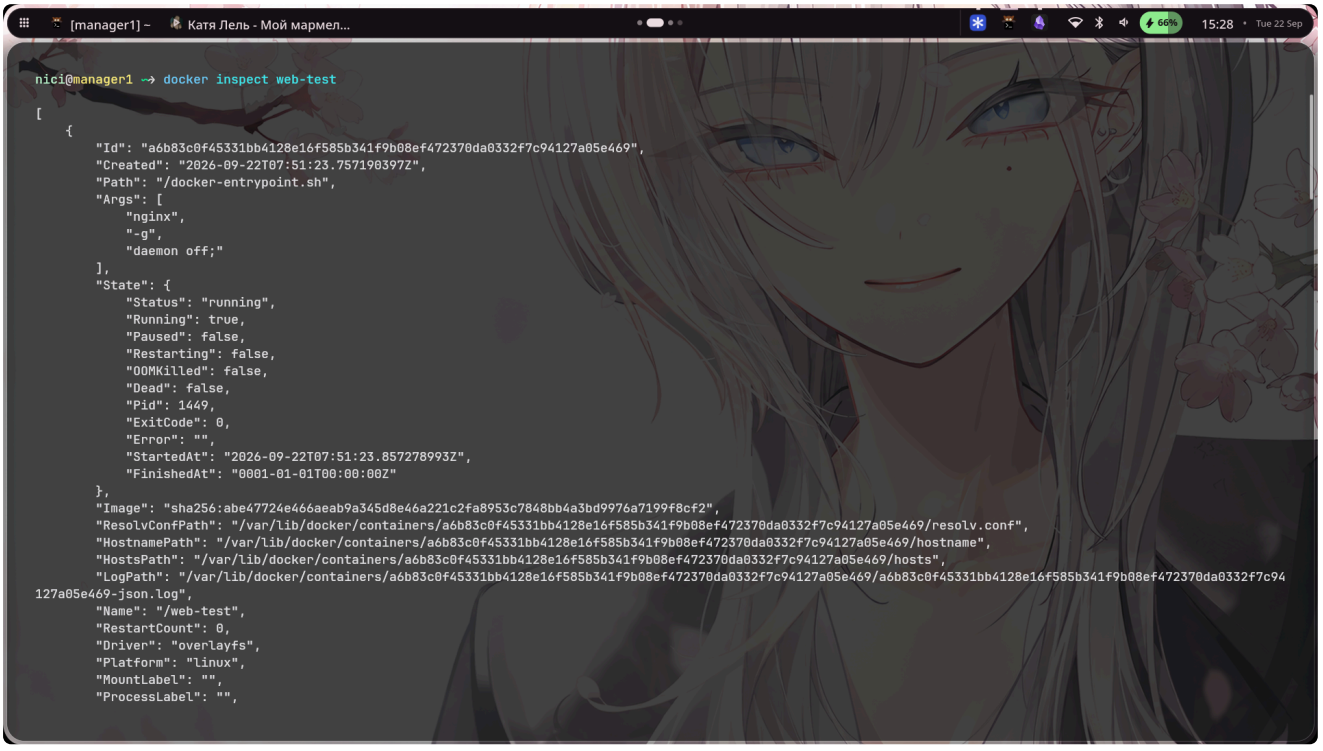
```
nici@manager1 ~> docker logs web-test

/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/22 07:51:24 [notice] 1#1: using the "epoll" event method
2026/09/22 07:51:24 [notice] 1#1: nginx/1.31.6
2026/09/22 07:51:24 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/09/22 07:51:24 [notice] 1#1: OS: Linux 6.8.0-139-generic
2026/09/22 07:51:24 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/09/22 07:51:24 [notice] 1#1: start worker processes
2026/09/22 07:51:24 [notice] 1#1: start worker process 29
nici@manager1 ~> docker inspect web-test
```

lalu kita juga akan melihat detail lengkap container dengan command

```
docker inspect web-test
```

yang mana hasilnya akan seperti,



lalu kita juga bisa monitoring resource container dengan command,

```
docker stats
```

yang mana akan menghasilkan output:

```
CONTAINER ID   NAME       CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O         PIDS
a6b83c0f4533   web-test   0.00%     16.59MiB / 1.904GiB   0.85%     1.67kB / 126B   32.1MB / 32.8kB   2
```

cukup itu dan kita akan beralih ke volume dan network, yang mana volume sendiri itu ibarat seperti ssd yang mana jika kita merakit pc lalu pc kita rusak, belum tentu data kita yang ada di dalam ssd itu hilang kecuali kalau kita format ulang dan volume yang kita buat akan selalu ada pada direktori /var/lib/docker/volumes/, kalau network sebenarnya itu hanyalah konfigurasi network antar container nya saja yang mana kurang lebih mirip dengan network di vm maupun di dunia nyata yang mana dunia nyata jauh lebih kompleks dari itu dan kita akan langsung praktek membuat sebuah volume dengan command

```
docker volume create data-web
```

lalu kita juga akan lihat volume kita dengan

```
docker volume ls
```

yang mana jika kedua perintah tersebut dijalankan maka akan menghasilkan

```
nici@manager1 ~> docker volume create data-web

data-web
nici@manager1 ~> docker volume ls
DRIVER    VOLUME NAME
local     data-web
local     f7fbbbc9d6cb1d4ff666648b8d24731bd1172a9d3f77a51231f7dd00fa11a4c5
nici@manager1 ~>
```

lalu kita akan coba untuk jalankan sebuah service nginx pada volume tersebut dengan perintah

```
docker run -d --name web-vol -v data-web:/usr/share/nginx/html nginx
```

lalu hasilnya seperti ini,

```
nici@manager1 ~> docker run -d --name web-vol -v data-web:/usr/share/nginx/html nginx

40f77dd9dc9b7f7196046e0a35f5d3556da0ec6afdc2bc4d5c5a5700393d82e4
nici@manager1 ~>
```

oke next kita akan coba buktikan kalau misal container dihapus, volume juga akan terhapus dan sebelum itu kita juga akan berikan suatu tulisan pada file nginx nya dengan command

```
docker exec web-vol sh -c "echo 'hello dari volume' >
/usr/share/nginx/html/test.txt"
```

dan kita coba remove container nya dengan command

```
docker rm -f web-vol
```

lalu kita akan coba buat baru containernya dengan volume yang sama dengan command

```
docker run -d --name web-vol2 -v data-web:/usr/share/nginx/html nginx
```

lalu kita juga akan tes apakah tadi 'hello dari volume' masih ada atau tidak dengan command

```
docker exec web-vol2 cat /usr/share/nginx/html/test.txt
```

lalu kira kira hasilnya seperti ini



disini bisa kita lihat bahwa volume masih akan tetap ada walaupun container terhapus dan kita akan ke network yang mana langsung saja kita akan praktek 3 jenis network pada docker, yaitu bridge yang mana container akan diberi ip sendiri yang terisolasi dari host, lalu host yang mana container itu numpang langsung dengan network host jadi kalau kita ngebuka port 80 di container, di host juga akan terbuka port 80, lalu none yang mana container hanya punya ip loopback, sama sekali gapunya akses ke network host dan langsung aja kita praktek bridge dengan command

```
docker run -d --name test-bridge nginx
```

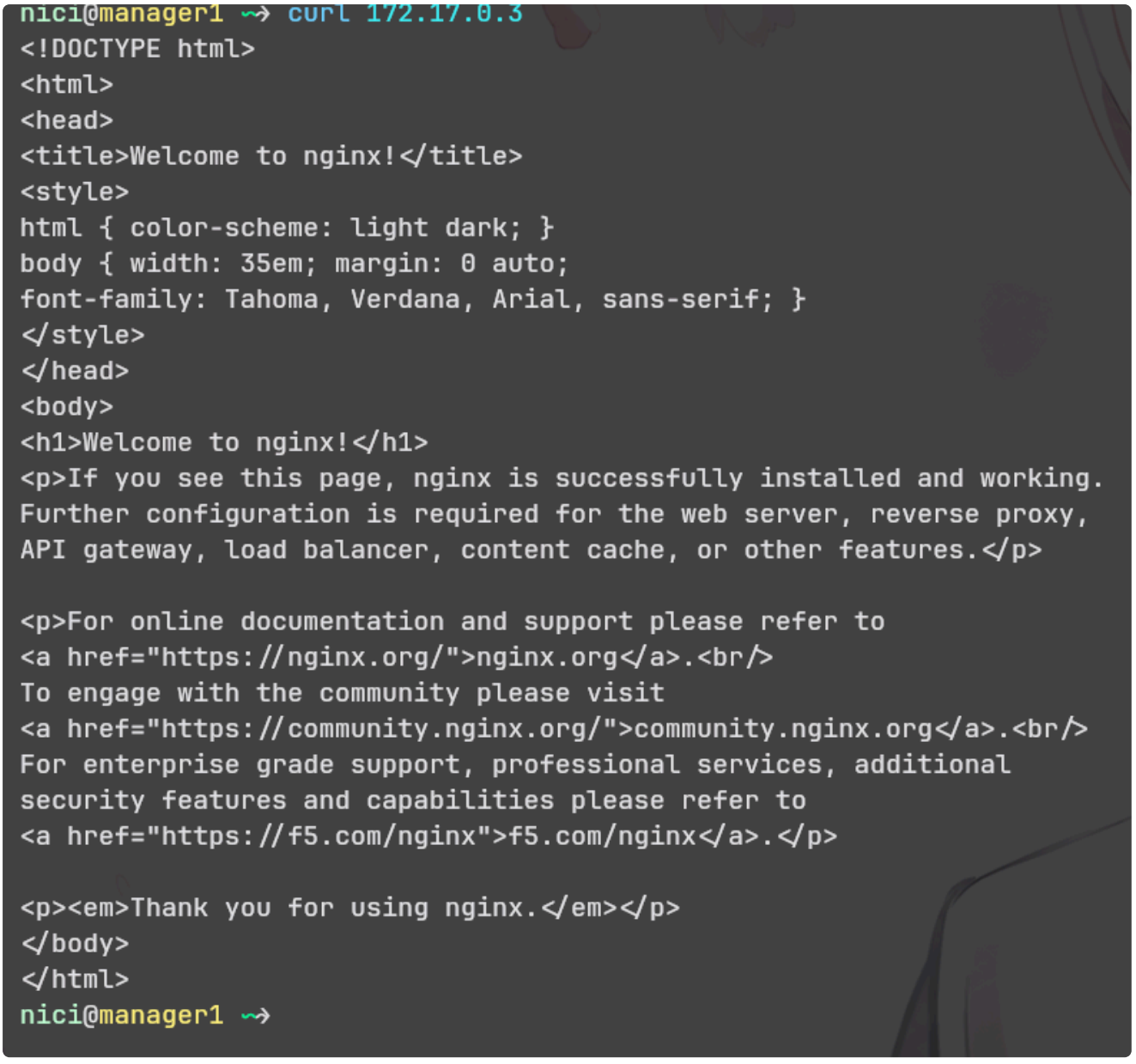
dan cek ip kita dengan command

```
docker inspect --format '{{.NetworkSettings.Networks.bridge.IPAddress}}'
test-bridge
```

yang mana menghasilkan output



lalu kita coba tes dengan curl ke ip yang keluar pakai command



nah disitu sudah bisa untuk yang bridge lalu kita akan coba pakai host dengan command

```
docker run -d --name test-host --network host nginx
```

lalu langsung kita tembak curl ke localhost karena kalau host itu seperti di awal container yang numpang langsung ke host jadi kalau container buka port 80 ya host juga buka dengan command

```
curl localhost:80
```

yang menghasilkan output:

```
nici@manager1 ~> docker run -d --name test-host --network host nginx

c3239a1aae9a3560249fcab37abaa1aa45c5c4dcac2613f057ae7b2b4e76c0ee
nici@manager1 ~> curl localhost:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
nici@manager1 ~>
```

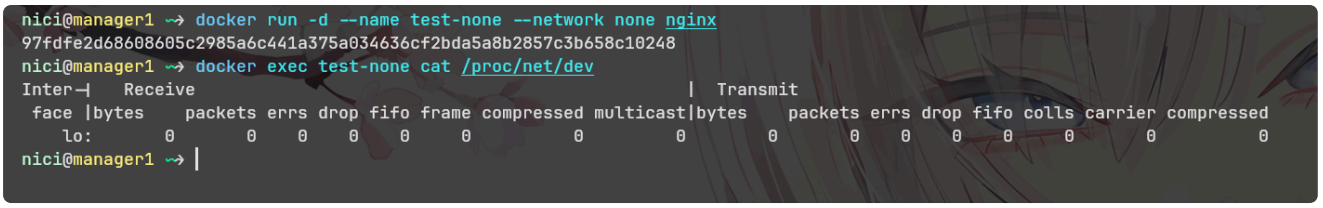
dan yang terakhir kita akan praktek none, ya ini kaya internal network di virtualbox dan langsung saja kita akan pakai command

```
docker run -d --name test-none --network none nginx
```

dan kita juga akan ngecek dengan command

```
docker exec test-none cat /proc/net/dev
```

yang mana outputnya seperti ini



nah udah bagian network dan volume lalu kita akan lanjut ke restart policy dan batas resource yang mana untuk restart itu sendiri intinya adalah kita ngatur apa yang terjadi kalau container mati dan ada 4 pilihan mode yang mana pertama adalah no, itu dimana saat container mati ya mati, ga ngapa ngapain lalu kedua ada always, nah ini always ini dimana container akan selalu ngerestart dengan use case kita matiin containernya dan saat kita restart docker daemon nya maka container yang udah mati ini akan nyala lagi lalu yang ketiga ada unless-stopped, ini mirip always tapiii kalau kita matiin manual dulu rebelum restart, maka containernya akan tetap mati tapi selain itu akan tetap nyala dan yang terakhir ada on-failure yang mana ini akan bisa restart kalau emang ada exit code yang nunjukin error, dan bukan exit yang manual seperti kita matiin containernya sendiri ya lalu ini juga bisa di limit mau berapa kali restart dulu dan kalau gagal ini container akan mati dan langsung aja kita akan coba unless-stopped dulu dengan command

```
docker run -d --name test-restart --restart unless-stopped nginx
```

lalu inspect dengan command

```
docker inspect --format '{{.HostConfig.RestartPolicy.Name}}' test-restart
```

yang mana akan menghasilkan output seperti ini,



disitu terlihat bahwa kita menggunakan policy unless-stopped dan lalu kita akan langsung aja praktek yang on-failure dengan command

```
docker run -d --name test-fail --restart on-failure:3 nginx sh -c "exit 1"
```

nah disini artinya adalah kita meng apply on-failure yang mana container itu akan exit dengan kode error exit 1 lalu dia akan mencoba restart tapi maksimal restart hanya 3 kali saja, setelah itu dia akan mati dengan status akhir nya adalah exited, bukan restarting terus, lalu kita akan main batas batasan resource yang mana langsung aja kita bisa membatasi container akan menggunakan berapa resource dari host kita seperti berapa core yang akan kita beri, berapa memori yang akan kita anggarkan ke containernya dan langsung aja kita akan praktek dengan command

```
docker run -d --name test-limit --memory="256m" --cpus="0.5" nginx
```

disitu kita ngerun container dengan nama test-limit yang memori(maksudnya ram) itu 256 mb dan cpu yang kita alokasikan sebesar 0.5 core(setengan core doang)

dan kita akan langsung inspeksi dengan command

```
docker inspect --format '{{.HostConfig.Memory}} bytes |
{{.HostConfig.NanoCpus}} nanocpus' test-limit
```

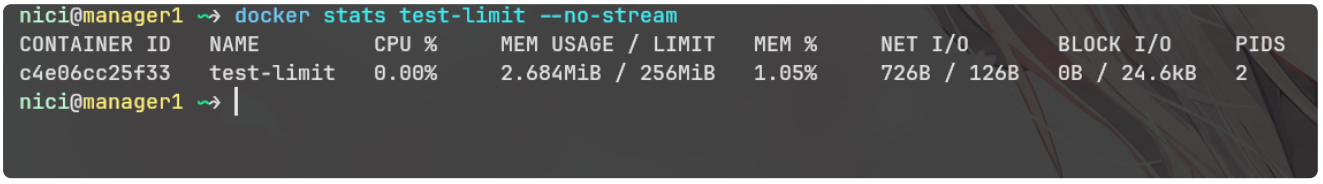
yang mana hasilnya adalah



kalau kurang jelas kita bisa pakai command

```
docker stats test-limit --no-stream
```

yang outputnya adalah



seperti itu dan kita disini akan langsung praktek untuk menjalankan 5 container yang berbeda dengan menjalankan command berikut secara berjamaah

```
docker run -d --name c1-nginx --memory="128m" --cpus="0.3" nginx

docker run -d --name c2-redis --memory="64m" --cpus="0.2" redis

docker run -d --name c3-postgres --memory="256m" --cpus="0.5" -e
POSTGRES_PASSWORD=test123 postgres

docker run -d --name c4-alpine --memory="32m" --cpus="0.1" alpine sleep
3600

docker run -d --name c5-busybox --memory="32m" --cpus="0.1" busybox sleep
3600
```

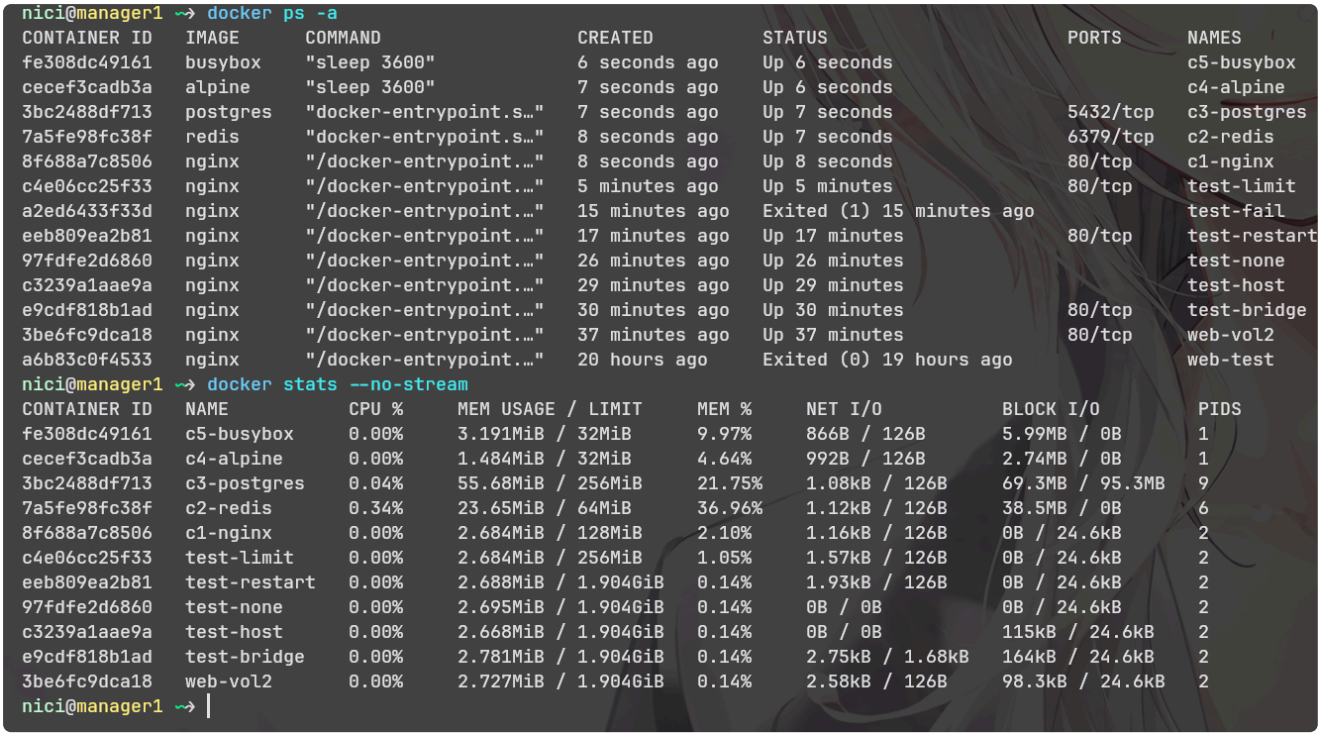
langsung aja hasilnya seperti ini jika kita melihat container dengan command

```
docker ps -a
```

lalu cek stats juga dengan command

```
docker stats --no-stream
```

yang output nya seperti ini,



```
nici@manager1 ~> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED          STATUS                          PORTS      NAMES
fe308dc49161   busybox    "sleep 3600"             6 seconds ago    Up 6 seconds                               c5-busybox
cecef3cadb3a   alpine     "sleep 3600"             7 seconds ago    Up 6 seconds                               c4-alpine
3bc2488df713   postgres   "docker-entrypoint.s…"   7 seconds ago    Up 7 seconds                    5432/tcp   c3-postgres
7a5fe98fc38f   redis      "docker-entrypoint.s…"   8 seconds ago    Up 7 seconds                    6379/tcp   c2-redis
8f688a7c8506   nginx      "/docker-entrypoint.…"   8 seconds ago    Up 8 seconds                    80/tcp     c1-nginx
c4e06cc25f33   nginx      "/docker-entrypoint.…"   5 minutes ago    Up 5 minutes                    80/tcp     test-limit
a2ed6433f33d   nginx      "/docker-entrypoint.…"   15 minutes ago   Exited (1) 15 minutes ago                  test-fail
eeb809ea2b81   nginx      "/docker-entrypoint.…"   17 minutes ago   Up 17 minutes                   80/tcp     test-restart
97fdfe2d6860   nginx      "/docker-entrypoint.…"   26 minutes ago   Up 26 minutes                              test-none
c3239a1aae9a   nginx      "/docker-entrypoint.…"   29 minutes ago   Up 29 minutes                              test-host
e9cdf818b1ad   nginx      "/docker-entrypoint.…"   30 minutes ago   Up 30 minutes                   80/tcp     test-bridge
3be6fc9dca18   nginx      "/docker-entrypoint.…"   37 minutes ago   Up 37 minutes                   80/tcp     web-vol2
a6b83c0f4533   nginx      "/docker-entrypoint.…"   20 hours ago     Exited (0) 19 hours ago                    web-test
nici@manager1 ~> docker stats --no-stream
CONTAINER ID   NAME           CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O         PIDS
fe308dc49161   c5-busybox     0.00%     3.191MiB / 32MiB      9.97%     866B / 126B       5.99MB / 0B       1
cecef3cadb3a   c4-alpine      0.00%     1.484MiB / 32MiB      4.64%     992B / 126B       2.74MB / 0B       1
3bc2488df713   c3-postgres    0.04%     55.68MiB / 256MiB     21.75%    1.08kB / 126B     69.3MB / 95.3MB   9
7a5fe98fc38f   c2-redis       0.34%     23.65MiB / 64MiB      36.96%    1.12kB / 126B     38.5MB / 0B       6
8f688a7c8506   c1-nginx       0.00%     2.684MiB / 128MiB     2.10%     1.16kB / 126B     0B / 24.6kB       2
c4e06cc25f33   test-limit     0.00%     2.684MiB / 256MiB     1.05%     1.57kB / 126B     0B / 24.6kB       2
eeb809ea2b81   test-restart   0.00%     2.688MiB / 1.904GiB   0.14%     1.93kB / 126B     0B / 24.6kB       2
97fdfe2d6860   test-none      0.00%     2.695MiB / 1.904GiB   0.14%     0B / 0B           0B / 24.6kB       2
c3239a1aae9a   test-host      0.00%     2.668MiB / 1.904GiB   0.14%     0B / 0B           115kB / 24.6kB    2
e9cdf818b1ad   test-bridge    0.00%     2.781MiB / 1.904GiB   0.14%     2.75kB / 1.68kB   164kB / 24.6kB    2
3be6fc9dca18   web-vol2       0.00%     2.727MiB / 1.904GiB   0.14%     2.58kB / 126B     98.3kB / 24.6kB   2
nici@manager1 ~>
```

perhatikan pada container container yang baru saja terbuat dan stats nya juga, lalu cheat sheet nanti ada di file .md yang terpisah aja lah ya. dan lanjut ke attack surface docker yang mana ini adalah hal yang tidak aku suka dari docker, yang mana tidak lain tidak bukan adalah adanya docker daemon disamping alasan personal ya disini aku akan jelaskan bagaimana cara sebuah container itu bisa mengatur segala sesuatu pada seluruh container hanya dengan container kita terkoneksi pada docker socket yang intinya gini, docker itu beda dengan podman yang mana tiap container itu ada init system nya sendiri, walau sama sama menggunakan linux containerisation, tapi secara arsitektur docker itu berbasis client-server yang mana sebenernya tiap kita ketik kata docker itu tuh clien saja dan dia cuma nembak request ke daemon yang jalan sebagai ROOT, sekali lagi ROOT dan daemon ini yang akan mengeksekusi segala perintah kita ke container yang masalahnya kita client itu ngobrol dengan daemon yang jalan sebagai root melalui socket dan siapa aja yang bisa akses ke socket itu maka hukumnya dia punya kuasa ke docker daemon wkwkwk, nah maka dari itu aku pribadi lebih suka container yang diberi init sendiri sendiri kaya podman daripada docker dan mari kita praktek bosskyuh yang mana langsung saja kita buat container nya langsung connect ke socket dengan command yang ada pada gambar dan

```
nici@manager1 ~> docker run -it --rm -v /:/host-root alpine chroot /host-root sh

# whoami
root
# cat /etc/shadow
root:*:20705:0:99999:7:::
daemon:*:20705:0:99999:7:::
bin:*:20705:0:99999:7:::
sys:*:20705:0:99999:7:::
sync:*:20705:0:99999:7:::
games:*:20705:0:99999:7:::
man:*:20705:0:99999:7:::
lp:*:20705:0:99999:7:::
mail:*:20705:0:99999:7:::
news:*:20705:0:99999:7:::
uucp:*:20705:0:99999:7:::
proxy:*:20705:0:99999:7:::
www-data:*:20705:0:99999:7:::
backup:*:20705:0:99999:7:::
list:*:20705:0:99999:7:::
irc:*:20705:0:99999:7:::
_apt:*:20705:0:99999:7:::
nobody:*:20705:0:99999:7:::
systemd-network:!*:20705::::::
systemd-timesync:!*:20705::::::
dhcpcd:!:20705::::::
messagebus:!:20705::::::
systemd-resolve:!*:20705::::::
pollinate:!:20705::::::
polkitd:!*:20705::::::
syslog:!:20705::::::
uuidd:!:20705::::::
tcpdump:!:20705::::::
tss:!:20705::::::
landscape:!:20705::::::
fwupd-refresh:!*:20705::::::
usbmux:!:20711::::::
```

lihat? kita bisa akses ke root hanya dengan akses ke docker socket melalui container dan satu lagi, kita bisa ngetik command

```
docker ps
```

dan hasilnya tebak

```
# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED             STATUS             PORTS      NAMES
979544c4fa9f   alpine     "chroot /host-root sh"   2 minutes ago       Up 2 minutes                  naughty_jepsen
fe308dc49161   busybox    "sleep 3600"             49 minutes ago      Up 49 minutes                 c5-busybox
cecef3cadb3a   alpine     "sleep 3600"             49 minutes ago      Up 49 minutes                 c4-alpine
3bc2488df713   postgres   "docker-entrypoint.s…"   49 minutes ago      Up 49 minutes      5432/tcp   c3-postgres
7a5fe98fc38f   redis      "docker-entrypoint.s…"   49 minutes ago      Up 49 minutes      6379/tcp   c2-redis
8f688a7c8506   nginx      "/docker-entrypoint.…"   49 minutes ago      Up 49 minutes      80/tcp     c1-nginx
c4e06cc25f33   nginx      "/docker-entrypoint.…"   54 minutes ago      Up 54 minutes      80/tcp     test-limit
eeb809ea2b81   nginx      "/docker-entrypoint.…"   About an hour ago   Up About an hour   80/tcp     test-restart
97fdfe2d6860   nginx      "/docker-entrypoint.…"   About an hour ago   Up About an hour              test-none
c3239a1aae9a   nginx      "/docker-entrypoint.…"   About an hour ago   Up About an hour              test-host
e9cdf818b1ad   nginx      "/docker-entrypoint.…"   About an hour ago   Up About an hour   80/tcp     test-bridge
3be6fc9dca18   nginx      "/docker-entrypoint.…"   About an hour ago   Up About an hour   80/tcp     web-vol2
#
```

yahaha kena tuh. dan kita lanjut ke docker bench security yang mana intinya ini kaya benchmark tapi konteks nya container security nya aja sih

```
docker run --rm --net host --pid host --userns host --cap-add
audit_control

-e DOCKER_API_VERSION=1.41

-v /etc:/etc\:ro

-v /usr/bin/containerd:/usr/bin/containerd\:ro

-v /usr/bin/runc:/usr/bin/runc\:ro

-v /usr/lib/systemd:/usr/lib/systemd\:ro

-v /var/lib:/var/lib\:ro

-v /var/run/docker.sock:/var/run/docker.sock\:ro

\--label docker_bench_security

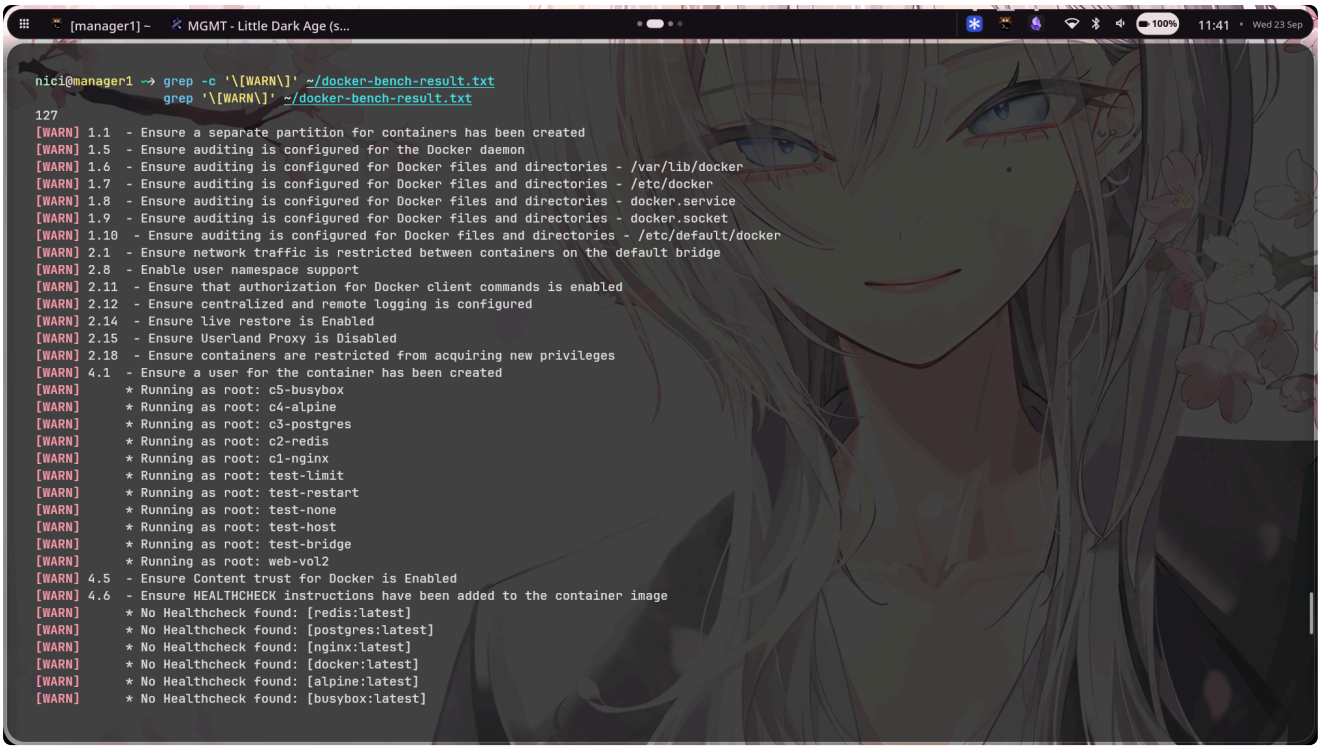
docker/docker-bench-security > \~/docker-bench-result.txt
```

jalanin itu aja dan cek dulu ringkasannya dengan command:

```
grep -c '[WARN]' \~/docker-bench-result.txt

grep '[WARN]' \~/docker-bench-result.txt
```

yang hasilnya akan seperti ini:



keluar tuh berbagai macam warning disana, dan sekian dari aku lalu terimakasih banyak